

Manual Técnico

Sistema de Venta de Boletos de Cine (Tipo Cinépolis)

Equipo de Desarrollo

28 de junio de 2026

Índice

1. Arquitectura	2
1.1. Visión general	2
1.2. Patrón MVC / arquitectura por capas	2
1.3. Flujo de una petición típica	4
1.4. Comunicación Frontend-Backend	4
1.5. Estructura y arquitectura del Frontend (rediseño por rol)	4
2. Tecnologías Utilizadas	6
3. Configuración	7
3.1. Requisitos previos	7
3.2. Clonar el proyecto	8
3.3. Estructura del repositorio (resumen)	8
3.4. Levantar MySQL con Docker	8
3.5. Instalación de dependencias del backend	8
3.6. Variables de entorno	9
3.7. Conexión a MySQL desde el backend	9
3.8. Configuración del Frontend	10
4. Ejecución	10
4.1. Orden de arranque	10
4.2. Entorno de desarrollo	10
4.3. Entorno de producción / demo	10
4.4. Verificación rápida	10
4.5. Detener el entorno	11
4.6. Flujo de trabajo recomendado para el equipo	11
5. Convenciones del Equipo	11
5.1. Convención de nombres	11
5.2. Reglas generales de código	12
5.3. Orden de construcción del proyecto	12

1. Arquitectura

1.1. Visión general

El sistema sigue una arquitectura **Cliente-Servidor desacoplada**, donde la comunicación entre el frontend y el backend se realiza exclusivamente mediante una **API RESTful** sobre HTTP, intercambiando datos en formato JSON.

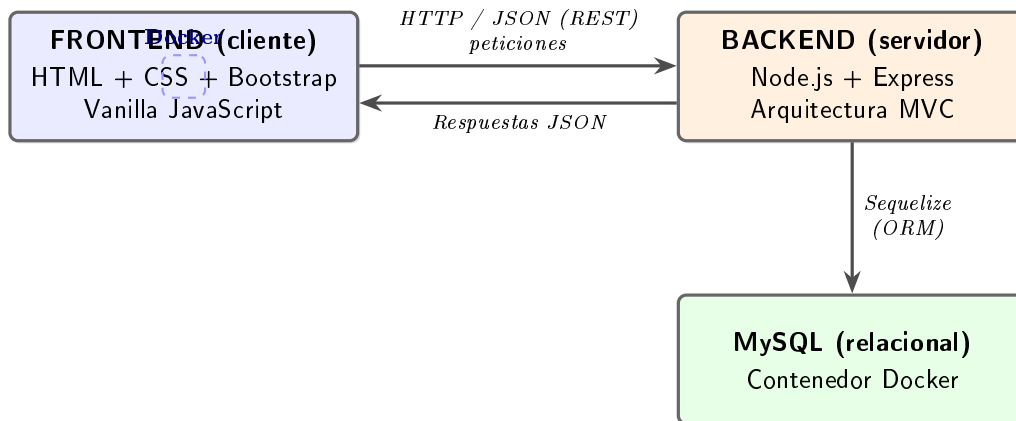


Figura 1: Comunicación Cliente-Servidor y persistencia en MySQL sobre Docker.

- El **frontend** (proporcionado por el docente) no accede directamente a la base de datos. Toda operación (login, consulta de películas, compra de boletos, dashboard administrativo) se realiza mediante `fetch()` hacia los endpoints REST expuestos por el backend.
- El **backend** expone recursos siguiendo convenciones REST (verbos HTTP GET, POST, PUT, DELETE) y responde en formato JSON con códigos de estado HTTP coherentes (200, 201, 400, 401, 403, 404, 500).
- La **autenticación** se maneja mediante JSON Web Tokens (JWT): el cliente envía el token en el header `Authorization: Bearer <token>` en cada petición protegida; un middleware del backend lo valida y determina el rol (`cliente` / `administrador`).
- La **base de datos** es **MySQL**, ejecutada dentro de un **contenedor Docker**, lo que permite que los tres integrantes del equipo trabajen con una instancia de base de datos idéntica y reproducible, sin depender de instalaciones locales distintas. Al tratarse de entidades con relaciones bien definidas (usuarios, películas, salas, asientos, funciones, compras, boletos), un modelo **relacional** es más adecuado que uno NoSQL: permite declarar claves foráneas, restricciones de integridad y evitar datos huérfanos (ej. una compra sin función asociada).

1.2. Patrón MVC / arquitectura por capas

El backend se organiza en capas con responsabilidades separadas, lo cual facilita el trabajo en equipo: cada integrante puede tomar un módulo sin interferir con el trabajo de los demás.

```

1 backend /
2 |
3 | -- src /
4 |   |-- config /                # Conexion a MySQL, variables globales
  
```

```

5 |         '-- db.js
6 |
7 |     |-- models/                # Modelos de Sequelize (capa "M")
8 |         |-- Usuario.model.js
9 |         |-- Pelicula.model.js
10 |        |-- Sala.model.js
11 |        |-- Asiento.model.js
12 |        |-- Funcion.model.js
13 |        |-- Compra.model.js
14 |        |-- Boleto.model.js
15 |        '-- index.js            # Inicializa Sequelize y define
    asociaciones
16 |
17 |    |-- routes/                 # Definicion de endpoints
18 |        |-- auth.routes.js
19 |        |-- peliculas.routes.js
20 |        |-- funciones.routes.js
21 |        |-- compras.routes.js
22 |        |-- boletos.routes.js
23 |        '-- admin.routes.js
24 |
25 |    |-- controllers/            # Reciben req/res, validan entrada (capa "C
    ")
26 |        |-- auth.controller.js
27 |        |-- peliculas.controller.js
28 |        |-- compras.controller.js
29 |        |-- boletos.controller.js
30 |        '-- admin.controller.js
31 |
32 |    |-- services/               # Logica de negocio pura
33 |        |-- compra.service.js   # validacion de asientos, descuento, etc
    .
34 |        |-- qr.service.js       # generacion de codigos QR
35 |        |-- pdf.service.js      # generacion de boletos y reportes PDF
36 |        '-- estadisticas.service.js
37 |
38 |    |-- middlewares/            # Interceptan la peticion antes del
    controlador
39 |        |-- auth.middleware.js   # verifica JWT
40 |        |-- role.middleware.js   # verifica rol (cliente/admin)
41 |        '-- error.middleware.js  # manejo centralizado de errores
42 |
43 |    |-- utils/                  # Helpers genericos
44 |
45 |    '-- app.js                  # Configuracion de Express
46 |
47 |    |-- server.js               # Punto de entrada del servidor HTTP
48 |    |-- docker-compose.yml      # Orquesta el contenedor de MySQL
49 |    |-- .env                    # Variables de entorno (NO se sube al
    repositorio)
50 |    |-- .env.example
51 |    |-- .gitignore
52 |    '-- package.json

```

1.3. Flujo de una petición típica

Ejemplo: compra de boletos.

1. El frontend hace `POST /api/compras` enviando el JWT en el header.
2. `auth.middleware.js` valida el token; `role.middleware.js` confirma que el usuario tiene rol cliente o administrador.
3. `compras.routes.js` enruta la petición hacia `compras.controller.js`.
4. El controlador valida el formato del cuerpo de la petición (cantidad de boletos, asientos seleccionados) y delega la lógica de negocio a `compra.service.js`.
5. El servicio verifica las reglas de negocio (asientos disponibles, capacidad de la sala, descuento por membresía), persiste el registro en MySQL mediante los `models` (Sequelize), genera el código QR (`qr.service.js`) y el PDF del boleto (`pdf.service.js`).
6. El controlador devuelve la respuesta JSON con los datos de la confirmación de compra.

1.4. Comunicación Frontend–Backend

- Todas las URLs de la API se centralizan bajo un prefijo común, por ejemplo `http://localhost:4000/api/...`
- El frontend (Vanilla JS) usa `fetch()` con `async/await`, parseando siempre la respuesta como JSON y manejando errores según el código de estado HTTP.
- CORS se habilita explícitamente en el backend (librería `cors`) para permitir que el frontend, servido desde otro origen/puerto, consuma la API durante el desarrollo.

1.5. Estructura y arquitectura del Frontend (rediseño por rol)

El frontend entregado originalmente por el docente (carpeta `UPAcine_front/`) llegó como una única página (`index.html` + `app.js` + `styles.css`) con todos los paneles –login, catálogo, asientos, pago, dashboard administrativo– ocultos/mostrados dentro del mismo documento. El equipo decidió **reorganizarlo en varias páginas HTML separadas por rol**, ya que cliente y administrador tienen flujos, permisos y vistas completamente distintos. El login sigue siendo único; lo que cambia es el destino después de autenticarse.

Nueva estructura de carpetas propuesta:

```

1 UPAcine_front/
2 |-- login.html           # Unico punto de entrada: login + registro
3 |-- cliente.html        # Catalogo, asientos, pago, boleto (rol "cliente")
4 |-- admin.html          # Dashboard, estadísticas, reportes (rol "
   adminadministrador")
5 |-- styles.css
6 |-- api.js              # fetch() centralizado hacia el backend (
   compartido)
7 |-- auth.js             # guarda/lee JWT y rol, protege paginas, hace
   el redirect
8 |-- cliente.js          # logica del flujo de compra (deriva del app.
   js original)
9 '-- admin.js            # logica del panel administrativo (tablas,
   Chart.js, PDF)

```

Responsabilidad de cada archivo:

- **login.html + auth.js**: formulario de inicio de sesión y registro. Al enviar el formulario, llama a `POST /api/auth/login`; el backend responde un JWT cuyo payload incluye `{ id, nombre, rol }`. **auth.js** guarda el token en `localStorage` y redirige según el rol recibido.
- **cliente.html + cliente.js**: contiene únicamente los paneles del cliente (catálogo de películas, selección de función/asientos, resumen con membresía, pago simulado, confirmación con QR, descarga de boleto en PDF). Reutiliza la lógica que ya existía en el **app.js** original (`renderSeats`, `toggleSeat`, `updateSummary`, `renderTicket`), pero acotada solo a este rol.
- **admin.html + admin.js**: contiene únicamente el dashboard administrativo (indicadores, gráficas con `Chart.js`, tabla de ventas, descarga de reportes en PDF). También reutiliza lógica del **app.js** original (`renderSalesTable`, `updateDashboard`, `renderChart`), separada del flujo de cliente.
- **api.js**: módulo común de acceso a datos (capa de *fetch*); lo consumen tanto **cliente.js** como **admin.js**, evitando duplicar código de llamadas HTTP.
- **auth.js**: además del login, expone una función de protección de ruta usada al cargar **cliente.html** y **admin.html**.

Flujo de autenticación y redirección por rol:

1. El usuario inicia sesión en **login.html**.
2. **auth.js** envía las credenciales a `POST /api/auth/login` y recibe el JWT.
3. Se guarda el token (y opcionalmente el rol) en `localStorage`.
4. Según el rol devuelto por el backend:

```
1 // auth.js
2 if (usuario.rol === "administrador") {
3   window.location.href = "admin.html";
4 } else {
5   window.location.href = "cliente.html";
6 }
```

5. Al cargar **cliente.html** o **admin.html**, cada página invoca al inicio `protegerRuta('cliente')` o `protegerRuta('administrador')` respectivamente: si no hay token válido o el rol no coincide, **auth.js** redirige de vuelta a **login.html**.

Ventajas de esta separación frente al **index.html** único original:

- Cada rol carga solo el HTML/JS que necesita (más ligero, menos paneles ocultos innecesarios).
- Permite repartir el trabajo del equipo de forma clara: una persona puede enfocarse en **cliente.js**, otra en **admin.js**, sin pisarse el código.
- El control de acceso por rol queda explícito a nivel de página, no solo escondiendo paneles con CSS.

Nota: esta reorganización es puramente de estructura/navegación; las reglas del proyecto permiten modificar la interfaz gráfica siempre que se conserven todas las funcionalidades solicitadas. La lógica de renderizado ya existente en el `app.js` original (mapa de asientos, resumen de pago, gráficas, etc.) se conserva, solo se redistribuye entre `cliente.js` y `admin.js`.

2. Tecnologías Utilizadas

Tecnología	Rol en el proyecto	Justificación
Node.js	Entorno de ejecución del backend	JavaScript en el servidor, asíncrono y no bloqueante; ideal para manejar múltiples peticiones concurrentes (consultas de disponibilidad de asientos, compras simultáneas).
Express	Framework web del backend	Minimalista y flexible; facilita definir rutas, middlewares y el ciclo petición/respuesta de forma ordenada, alineado al patrón MVC.
MySQL	Base de datos relacional	El dominio del problema (usuarios, películas, salas, asientos, funciones, compras, boletos) tiene relaciones fuertes y bien definidas (claves foráneas, integridad referencial); es exactamente el caso de uso ideal para una base de datos relacional, evitando datos huérfanos o inconsistentes.
Docker	Contenerización de la base de datos	Permite que los tres integrantes del equipo levanten una instancia idéntica de MySQL con un solo comando, evitando problemas de “ <i>en mi máquina sí funciona</i> ” y facilitando la entrega/despliegue del proyecto.
Sequelize	ORM (Object-Relational Mapping)	Define los modelos (<code>Usuario</code> , <code>Pelicula</code> , <code>Compra</code> , etc.) como clases JS, declara las relaciones (1 a N entre <code>Sala</code> → <code>Asientos</code> , <code>Pelicula</code> → <code>Funciones</code> , etc.) y traduce automáticamente a SQL, reduciendo errores de sintaxis manual y facilitando la sincronización del esquema entre los tres integrantes.
mysql2	Driver de bajo nivel	Utilizado internamente por Sequelize para la conexión real con MySQL; soporta promesas, necesario para <code>async/await</code> .

Tecnología	Rol en el proyecto	Justificación
jsonwebtoken (JWT)	Autenticación basada en roles	Permite emitir un token firmado al iniciar sesión, conteniendo el <code>id</code> y <code>rol</code> del usuario; evita manejar sesiones en servidor (stateless), ideal para una API REST.
bcrypt / bcryptjs	Hash de contraseñas	Nunca se almacenan contraseñas en texto plano; se guarda el hash con salt, cumpliendo buenas prácticas de seguridad.
cors	Middleware de Express	Habilita que el frontend (servido en otro puerto/origen) realice peticiones al backend sin ser bloqueado por la política de same-origin del navegador.
dotenv	Manejo de variables de entorno	Separa configuración sensible (credenciales de MySQL, puerto, JWT secret) del código fuente, leyéndola desde <code>.env</code> .
qrcode	Generación de códigos QR	Requisito del proyecto: cada boleto debe incluir un QR generado automáticamente, codificando el folio único de la compra.
pdfkit	Generación de documentos PDF	Requisito del proyecto: boletos descargables y reportes administrativos (ventas, ingresos, por película, por día) deben entregarse en PDF.
express-validator	Validación de datos de entrada	Valida los datos del formulario de pago simulado (16 dígitos de tarjeta, CVV, fecha de vencimiento) y otros inputs antes de llegar a la lógica de negocio.
nodemon	Herramienta de desarrollo	Reinicia automáticamente el servidor al detectar cambios en el código, agilizando el ciclo de desarrollo del equipo.
Chart.js (<i>frontend</i>)	Visualización de estadísticas	Recomendado para graficar ventas por día, por película, por sala, etc., en el dashboard administrativo.

3. Configuración

3.1. Requisitos previos

Cada integrante del equipo debe tener instalado:

- **Node.js** (versión LTS, ≥ 18)
- **Docker** y **Docker Compose**
- **Git**
- Un editor de código (VS Code recomendado)

3.2. Clonar el proyecto

```
1 git clone <url-del-repositorio>
2 cd <nombre-del-proyecto>
```

3.3. Estructura del repositorio (resumen)

```
1 proyecto-cinopolis/
2 |-- backend/      # API REST (Node.js + Express + MySQL)
3 |-- frontend/     # Interfaz proporcionada por el docente
4 |-- Documento/    # Manual tecnico e investigacion
5 '-- README.md
```

3.4. Levantar MySQL con Docker

En la raíz de `backend/` se incluye un archivo `docker-compose.yml` que define el contenedor de MySQL:

```
1 version: "3.8"
2
3 services:
4   mysql:
5     image: mysql:8
6     container_name: cinopolis_mysql
7     restart: unless-stopped
8     ports:
9       - "3306:3306"
10    environment:
11      MYSQL_ROOT_PASSWORD: admin123
12      MYSQL_DATABASE: cinopolis_db
13      MYSQL_USER: cinopolis_user
14      MYSQL_PASSWORD: cinopolis_pass
15    volumes:
16      - mysql_data:/var/lib/mysql
17
18 volumes:
19   mysql_data:
```

Para levantar el contenedor:

```
1 docker compose up -d
```

Verificar que el contenedor esté corriendo:

```
1 docker ps
```

3.5. Instalación de dependencias del backend

```
1 cd backend
2 npm install
```

Esto instala, entre otras, las dependencias declaradas en `package.json`:


```
1 npm install express sequelize mysql2 jsonwebtoken bcryptjs cors dotenv
  qrcode pdfkit express-validator
2 npm install --save-dev nodemon
```

3.6. Variables de entorno

Crear un archivo `.env` en `backend/` (basado en `.env.example`, el cual sí se sube al repositorio):

```
1 # Servidor
2 PORT=4000
3
4 # Base de datos (MySQL en Docker)
5 DB_HOST=localhost
6 DB_PORT=3306
7 DB_NAME=cinepolis_db
8 DB_USER=cinepolis_user
9 DB_PASSWORD=cinepolis_pass
10
11 # Autenticacion
12 JWT_SECRET=una_clave_secreta_larga_y_unica
13 JWT_EXPIRES_IN=8h
```

Importante: el archivo `.env` *nunca* debe subirse al repositorio (debe estar listado en `.gitignore`), ya que contiene credenciales. Cada integrante puede usar las mismas credenciales del `docker-compose.yml` al trabajar en local, garantizando un entorno idéntico para todo el equipo.

3.7. Conexión a MySQL desde el backend

```
1 // src/config/db.js
2 const { Sequelize } = require("sequelize");
3
4 const sequelize = new Sequelize(
5   process.env.DB_NAME,
6   process.env.DB_USER,
7   process.env.DB_PASSWORD,
8   {
9     host: process.env.DB_HOST,
10    port: process.env.DB_PORT,
11    dialect: "mysql",
12    logging: false,
13  }
14 );
15
16 const conectarDB = async () => {
17   await sequelize.authenticate();
18   console.log("Conexion a MySQL establecida correctamente.");
19 };
20
21 module.exports = { sequelize, conectarDB };
```

Los modelos (`models/`) se sincronizan contra MySQL al levantar el servidor (`sequelize.sync()`) o mediante migraciones, según se acuerde en el equipo.

3.8. Configuración del Frontend

Si el frontend se sirve de forma independiente (por ejemplo, con **Live Server** de VS Code), debe apuntar su capa de consumo de API (`api.js` o equivalente) a la URL base del backend:

```
1 const API_BASE_URL = "http://localhost:4000/api";
```

4. Ejecución

4.1. Orden de arranque

1. Levantar el contenedor de MySQL (una sola vez, queda corriendo en segundo plano):

```
1 docker compose up -d
```

2. Levantar el backend en modo desarrollo.
3. Abrir el frontend en el navegador.

4.2. Entorno de desarrollo

Dentro de `backend/`, en `package.json`:

```
1 "scripts": {  
2   "start": "node server.js",  
3   "dev": "nodemon server.js"  
4 }
```

Levantar el servidor en modo desarrollo (con reinicio automático ante cambios):

```
1 npm run dev
```

La consola debe confirmar:

```
1 Servidor corriendo en http://localhost:4000  
2 Conexion a MySQL establecida correctamente.
```

4.3. Entorno de producción / demo

```
1 npm start
```

4.4. Verificación rápida

- Probar un endpoint público, por ejemplo GET `http://localhost:4000/api/peliculas`, desde el navegador o Postman, para confirmar que el backend responde correctamente antes de integrar con el frontend.
- Verificar que el frontend (abierto en el navegador) pueda consumir dicho endpoint sin errores de CORS.
- Verificar con `docker ps` que el contenedor de MySQL sigue activo mientras el backend está en ejecución.

4.5. Detener el entorno

```
1 docker compose down
```

Esto detiene el contenedor de MySQL. Los datos persisten gracias al volumen `mysql_data` definido en `docker-compose.yml`, por lo que no se pierden al reiniciar.

4.6. Flujo de trabajo recomendado para el equipo

1. Cada integrante trabaja en una rama (`feature/auth`, `feature/compras`, `feature/dashboard`) y abre *pull requests* hacia `main` o `develop`.
2. Antes de programar, revisar este manual y el modelo de datos (`Usuario`, `Pelicula`, `Sala`, `Asiento`, `Funcion`, `Compra`, `Boleto`) para mantener consistencia entre módulos.
3. Probar cada endpoint de forma aislada (Postman/Thunder Client) antes de integrarlo con el frontend.
4. Todo el equipo trabaja contra la misma definición de `docker-compose.yml`, evitando discrepancias de entorno de base de datos.

5. Convenciones del Equipo

Esta sección existe para evitar el problema más común en proyectos de equipo: que cada integrante nombre las cosas distinto y el código termine *"no hablando el mismo idioma"* entre capas. Las reglas siguientes son obligatorias para todo el código del proyecto, sin excepción.

5.1. Convención de nombres

Elemento	Convención	Ejemplo
Variables y funciones (JS)	camelCase	<code>precioTotal</code> , <code>obtenerPeliculas()</code>
Modelos / Clases	PascalCase	<code>Usuario</code> , <code>Compra</code> , <code>Boleto</code>
Archivos de backend	kebab-case con sufijo de capa	<code>compra.controller.js</code> , <code>auth.middleware.js</code>
Archivos de frontend	kebab-case simple	<code>cliente.html</code> , <code>api.js</code>
Tablas de MySQL	plural, minúsculas	<code>usuarios</code> , <code>compras</code> , <code>boletos</code>
Rutas de la API	plural, minúsculas, en inglés o español (decidir una sola vez)	<code>/api/peliculas</code> , <code>/api/compras</code>
Variables de entorno	MAYUSCULAS_CON_GUION	<code>DB_PASSWORD</code> , <code>JWT_SECRET</code>

Regla de oro: el mismo campo, el mismo nombre en todas las capas. Si una columna se llama `peliculaId` en el modelo de Sequelize, debe llamarse igual en el JSON que devuelve el controlador y en la variable que lo recibe en `cliente.js`. Mezclar `movie_id` en un lado y `idPelicula` en otro es la causa más común de bugs por `undefined`.^{en} integraciones de equipo.

5.2. Reglas generales de código

- Nombres descriptivos y completos: `calcularDescuentoMembresia()`, no `calcDesc()` o `fn1()`.
- Nada de variables globales sueltas; todo vive dentro de su módulo/archivo correspondiente (por eso la separación en `routes/controllers/services/models`).
- Un archivo, una responsabilidad: si `compra.controller.js` empieza a tener lógica de cálculo de precios, esa lógica se mueve a `compra.service.js`.
- Antes de crear una función o variable nueva, buscar en el proyecto si ya existe algo similar con otro nombre (`Ctrl+Shift+F` en VS Code) para no duplicar conceptos.

5.3. Orden de construcción del proyecto

El desarrollo no se hace "de afuera hacia adentro" (empezando por la pantalla), sino al revés: primero se fija el dato, luego la regla de negocio, y al final la pantalla que lo muestra.

1. **Modelo de datos.** Definir las entidades y sus relaciones (`Usuario`, `Pelicula`, `Sala`, `Asiento`, `Funcion`, `Compra`, `Boleto`) en `models/`. Todo lo demás depende de esto, por lo que se acuerda en equipo **antes** de escribir cualquier otra línea de código.
2. **Capa de servicio (reglas de negocio).** Implementar la lógica pura en `services/` (ej. validar asientos disponibles, aplicar el 20 % de descuento, generar QR/PDF) sin mezclarla con detalles de HTTP.
3. **Controladores.** Conectan la petición HTTP (`req/res`) con el servicio correspondiente; validan el formato de entrada.
4. **Rutas.** Mapean cada URL + verbo HTTP a su controlador.
5. **Pruebas del endpoint de forma aislada** (Postman/Thunder Client), **antes** de tocar el frontend.
6. **Contrato de la API.** Documentar (aunque sea informalmente, en un `README` o tablero compartido) qué recibe y qué devuelve cada endpoint, para que quien programe el frontend no tenga que adivinar nombres de campos.
7. **Frontend.** Se conecta al final: `api.js` (consumo) → lógica de la página (`cliente.js` / `admin.js`) → integración visual con lo ya entregado.
8. **Integración y pruebas conjuntas** de todo el flujo (login → compra → confirmación → dashboard).

Por qué este orden importa: si se empieza por el frontend, se acaban inventando nombres de campos y estructuras de datos .^a ojo"que luego no coinciden con lo que realmente entrega el backend, obligando a reescribir código de ambos lados. Fijar primero el modelo y el contrato de la API evita ese retrabajo.